

黄渡理工大学 × 同济大学

百年
修得同船渡
未来也要一起走

实验报告名称: 《高级语言程序设计》课程大作业2——数织. 实验报告

班级: 5000244001602

学号: 2452098

姓名: 赵崇治

完成日期: 2025年6月12日

1. 数织游戏以及相关背景

1.1. 题目

题目可以粗略地分为两个部分，分别是子题1-3/4-11。其中第一部分需要实现内部数组的初始化，并初步打印出一个看得见的界面，并且玩家可以通过键盘输入行列完成交互。第二部分需要利用已有tools函数，实现两种不同框架下伪图形界面绘画，形成一个相对合理而完整的游戏，玩家可以通过鼠标点击行列完成交互。

第一部分的码量主要放置在子题3中，一方面是因为需要实现根据每一次玩家的输入实时刷新整个棋盘，然后往下不断输出；另一方面，在Update 1中，需要对作弊模式实现可持久化，并且根据相关内部逻辑实现切换作弊前后的打印正确。

第二部分的码量主要放置在边框绘画的函数上，且每道子题都投入相对均衡的时间。一方面，虽然4-7(无分隔线版)绘画起来相对简单，但需要首次实现鼠标的坐标交互，实时显示所在行列以及作弊模式；另一方面，8-11(有分隔线版)绘画起来相对复杂，需要重新绘制各种内部外部线条，但内部交互等可以通过改写先前函数实现复用共用。

共用函数中基本原则就是高效完成程序，减少冗余代码。函数分解合理规范的，可以给予最多1分的总份额外加分。

在初版作业要求中，内部数组的数据类型、字体、字号、颜色等无强制要求，但需要保证在1920x1080分辨率/缩放率100%条件下显示正常。不过值得注意的是在Update1中，对于作弊模式的颜色显示进行了一定的要求，以很大程度上便利了程序的调试。

边框的线型无强制要求(四选一)，但必须是中文边框线(“附件四种线型的中文制表符框架.cpp”给出了四种线型，任选一种即可)。各种提示信息、状态栏的内容等无强制要求出错时的各种提示无强制要求，清晰明了即可。

1.2. 背景

数织(Nonogram)，是一种逻辑游戏，以猜谜的方式绘画黑白位图。在一个网格中，每一行和列都有一组数，玩家需根据它们来填满或留空格子。

在1987年，日本人西尾彻也发明了这个游戏，日文名称是“お絵かきロジック”，意思是“绘画逻辑”，这是因为发明最初，玩家填满或留空格子，直到最后就可以由此得出一幅图画。

2. 整体设计思路

本程序的设计以模块化和可扩展性为核心，围绕一个GameState结构体来管理游戏的所有状态数据，包括谜题数据、玩家进度、行列提示以及作弊模式开关等。整体程序流程分为三大模块：

主控与菜单模块（main 函数, show_menu 函数）：程序从main函数启动，进入一个循环。在循环的开始，调用show_menu()函数 显示所有可选的游戏模式（A-K）以及退出选项（Q）。该菜单等待用户通过键盘输入一个有效选项，并将选择传递给下一步。当任何一个游戏模式结束后，程序会再次返回菜单，直到用户选择退出，程序结束。

游戏初始化模块（init_game, generate_balls, calculate_hints）：当用户选择一个游戏模式并输入期望的棋盘大小后（如5x5, 10x10, 15x15），程序开始初始化。首先，init_game函数设置GameState结构体中的行列数、清空棋盘和玩家进度数组。接着，generate_balls函数在棋盘上随机生成“谜底”。最后，calculate_hints函数根据生成的谜底计算出每行和每列的提示数字串。这一模块为所有游戏模式提供了统一的数据基础。

游戏循环与交互模块（以 loop_BIG 函数为例）：这是程序的核心部分。在进入此模块前，程序会先调用draw_fake_GUI函数绘制出完整的游戏界面框架，包括行列提示区和棋盘网格。随后，程序进入loop_BIG函数，这是一个事件驱动的循环。该循环持续监听键盘和鼠标事件，并根据事件类型（如鼠标移动、左键点击、右键点击、键盘按键）执行相应的操作，如更新光标位置显示、标记或取消标记方格、切换作弊模式、提交解答或退出游戏。所有操作都直接在图形界面上实时反馈，直到玩家解出谜题或主动退出。

3. 主要功能的实现

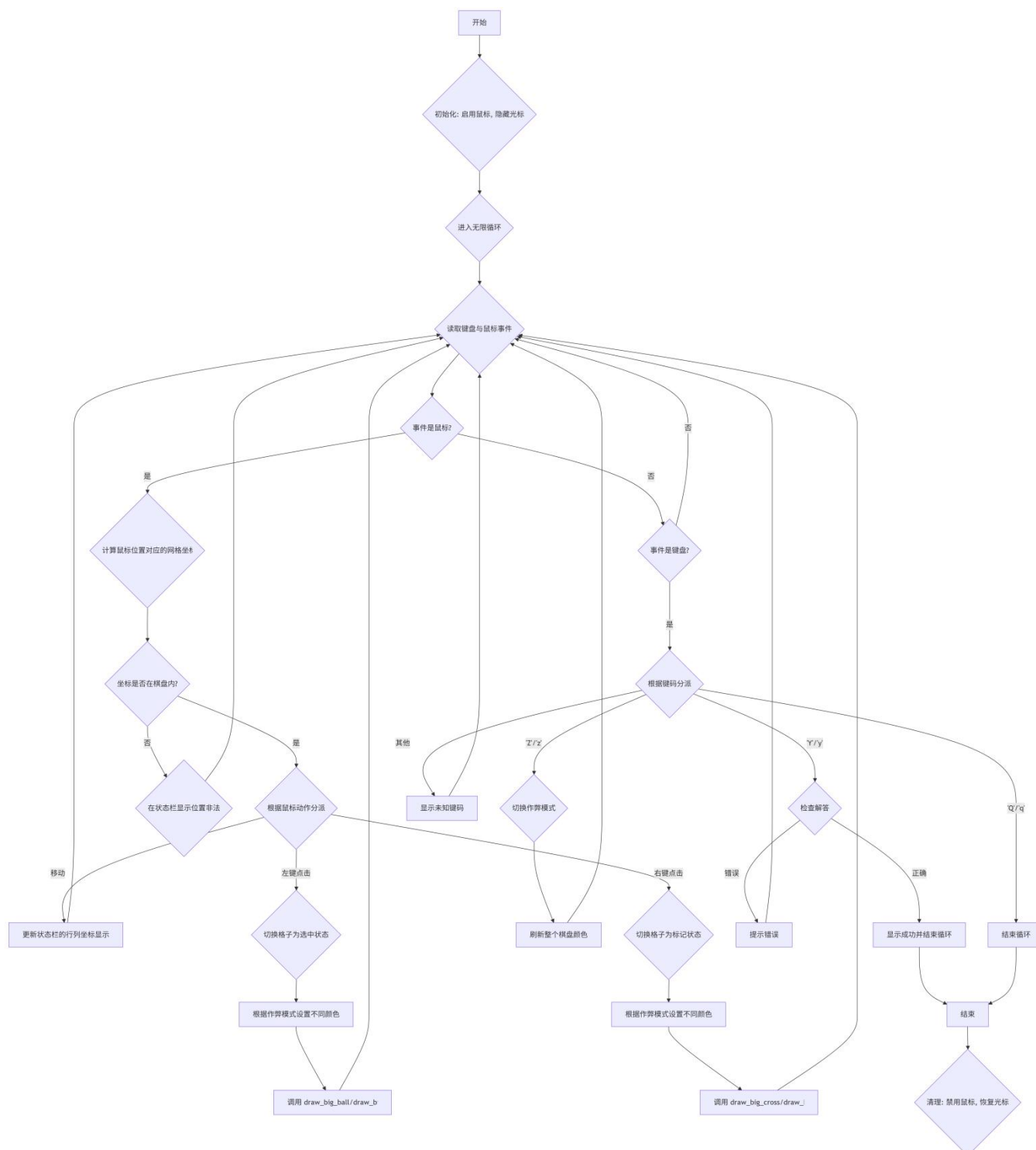
为了判断鼠标点击在哪个格子里，需要将屏幕的绝对坐标(mx, my) 转换为棋盘的相对网格坐标(grid_x, grid_y)。这通过精确计算实现：首先减去棋盘左上角的偏移量（由提示区的宽度 hint_cols 和高度 hint_rows 决定），然后除以每个格子所占的字符宽度（8 个字符）和高度（4 个字符）。代码中还特别处理了鼠标在格子边框上的点击，通过取模运算 (%) 判断点击是否在有效的单元格内部区域内。

按下 'Z' 键可以切换 game.cheat_mode 的布尔值。

在作弊模式下，玩家的每一次点击（左键或右键）都会立即与谜底(game.grid)进行比较。如果操作正确（例如，在有谜底的格子上画圈），则使用高亮蓝色（COLOR_HBLUE）；如果操作错误（例如，在没有谜底的格子上画圈），则使用高亮红色（COLOR_HRED）。

当从普通模式切换到作弊模式时，程序会遍历整个棋盘，根据 `game.player` 和 `game.grid` 的六种组合状态（如：有答案但未选、有答案且选对、无答案但错选等），为每个格子重新着色并刷新显示。这为玩家提供了全面的正确性参考。

当玩家按下 'Y' 键提交时，程序调用 `check(game)` 函数。该函数会重新计算一遍当前玩家棋盘 (`game.player`) 的行列提示数，并将其与谜底的真实提示数 (`game.row_hints`, `game.col_hints`) 进行逐一比对。如果完全一致，则判定成功并退出游戏循环；否则，提示错误。



游戏的核心是一个循环体，在一开始调用 `cct_read_keyboard_and_mouse()` 函数，阻塞式地等待用户输入。该函数能同时捕获键盘和鼠标的动作，并将事件类型(`ret`)、鼠标坐标(`mx`, `my`)、鼠标动作(`maction`)和键码(`keycode1`, `keycode2`) 返回。这种机制避免了 CPU 空转，且能及时响应用户操作。

4. 调试过程碰到的问题

1. 首先是子题3的缓冲区刷新，我一开始就是大群里说的用`gotoxy`来显示内部棋盘，给我带来了极大的困扰，不知道为啥demo每次计算的位置这么精确，能显示不错位。

解决：然后我看有同学也是遇到类似的问题，在大群里问，老师说“这样做没必要啊”我就明白了，前三个子题实际上都不需要利用`gotoxy`函数的输入，直接输出空格或者0/X整个随着边框一起往下打印就可以了。

2. 其次是游戏状态未正确重置。在一局游戏结束后，不退出程序，直接开始新的一局时，偶尔会发现新棋盘残留着上一局的玩家数据或作弊状态。

解决：在主while循环的开始，即`show_menu()`之后，重新初始化一个全新的GameState，确保在每次进入游戏前，所有成员（数组、标志位等）都被重置，从而隔离了每局游戏的状态。

3. 再其次是带有分隔线的棋盘（子题K）绘制困难，且很难对齐。`draw_fake_GUI`函数在绘制带分隔线的棋盘时，由于提示区大小动态变化，导致内外框线、`┌`、`┐`等特殊字符经常无法对齐。

解决：将绘制过程分解。先根据`hint_cols`和`hint_rows`计算出所有主要区域的起始坐标。然后，分步绘制：先画最外层边框，再画行列数字标签，最后循环绘制内部的行列分隔线。在循环中，精确计算每个`┌`、`┐`、`└`的位置，确保它们与动态的行列数适配。这需要大量的耐心和`gotoxy`的精确定位。

4. 最后是带有分隔线的棋盘（子题K）的边框判断方法，一开始想了很久没想到有没有除了直接判断所在行列xy值以外的方法，因为这样暴力判断实际上需要大量的重复计算，意思就是说，每次获取鼠标所在原始行列后，都需要额外计算是否与边框的坐标发生重叠。而内部边框又特别多，很难计算出来。

解决：通过逆向判断，将“检测是否点击在边框上”的复杂问题，简化为“检测是否点击在单元格的有效内容区内”的简单问题。（一开始自己就是想复杂了）

首先，为棋盘中的每个单元格定义一个固定的逻辑区域大小。从代码中可以看出，每个单元格在计算时被视为一个 `8x4` 的字符矩形区域。

当鼠标点击事件发生时，在通过除法计算出点击落在哪一个宏观的单元格 (`grid_x`, `grid_y`) 之后，再利用取模计算出点击位置在该`8x4`逻辑区域内的相对坐标 (`cell_x`, `cell_y`) 。

而在每个`8x4`的绘制区域内部，实际绘制内容的区域并没有占满。因此，我定义了一个比`8x4`稍小的“有效内容区” `6x3`的。

如此一来，判断逻辑就变得非常简单。如果鼠标点击后计算出的相对坐标 (`cell_x`, `cell_y`) 在这个 6x3 的有效区域内，则视为一次有效的单元格点击并进行处理。如果坐标超出了这个范围，程序就判定该次点击落在了单元格之间的边框或间隙上，直接不予响应即可。

这种方法巧妙地避开了去计算每一条横竖边框具体坐标的繁琐工作，仅通过一次取模和一次范围比较就完成了判断，逻辑清晰且效率极高。

5. 心得体会

5.1 完成本次作业得到的一些心得体会、经验教训

完成这次数织游戏的设计与实现，一个心得是构建一个看似简单的控制台“鼠标交互——图形界面”应用，其复杂度和工作量远超预期，考验了逻辑分解、流程控制和细节处理的能力。

经验上，我深刻体会到“前期设计优于后期调试”。在编写 `loop_BIG` 函数之前，花时间用伪代码和流程图梳理清楚事件处理的逻辑，远比直接上手编码然后陷入无尽的 bug 修复要高效。特别是对于作弊模式中复杂的条件判断，事先的逻辑梳理是成功的关键。

关于非 5/10/15 行列的适配问题，这一个部分应该从作业的角度来说不是强制要求，但是我吸取教训，为了程序的健壮性和未来的可扩展性，所有的核心数据处理函数，例如 `init_game`、`generate_balls` 以及 `calculate_hints`，在进行循环或数组访问时，都统一使用 `game.rows` 和 `game.cols` 这两个变量作为边界。代码中没有 5、10 或 15 之类的“魔法数字”，也就让程序和尺寸解耦。

在绘图形界面时，所有的绘图函数，如 `draw_frame` 和 `draw_fake_GUI`，都尽量接收结构体作为参数。函数内部会根据 `game.rows` 和 `game.cols` 的值动态计算需要绘制的 `—` 字符数量、`|` 字符数量以及提示区的宽高，确保无论棋盘尺寸如何变化，整体的框架结构都能按比例正确生成。

教训方面，在完成本次综合题的过程中，对于前后小题的关联关系还是略微欠缺，例如，子题 D/H 要求绘制一个静态框架 (`draw_frame`)，子题 E/I 在其基础上增加了提示数字的绘制 (`draw_fake_GUI`)，而最终的 G/K 则是在这个 GUI 基础上加入了事件循环 (`loop_SMALL/loop_BIG`)。这种渐进式的结构希望后面的小題可以复用前面小題已完成的绘制代码，只需添加新的交互功能即可。但由于精力有限，笔者没有完全复用所有能复用的代码/函数。

不过还是将程序的核心逻辑（如游戏初始化 `init_game`、谜底生成 `generate_balls`、提示计算 `calculate_hints`、答案校验 `check`）都封装在 `pullze_base.cpp` 中。这些函数不涉及任何界面打印，是纯粹的数据处理。因此，无论是前期的纯文本版（子题 1-3），还是后期的图形版（子题 G/K），都共享这套核心逻辑，无需重复编写。

后期小題是前期小題的有效扩展。将菜单显示 (`show_menu`)、获取尺寸输入 (`get_size_input`) 等通用功能封装在 `pullze_tools.cpp` 中，供 `main` 函数统一调用，避免了代码的重复。

如何才能更好地重用代码？首先设计之初就要分析需求中哪些是核心且不变的（如游戏规则、数据结构），哪些是易变的（如界面表现、交互方式）。将“不变”的部分封装成稳定的底层API，将“可变”的部分作为上层实现，两者通过清晰的接口调用。

让每个函数只做一件事。如calculate_hints只负责计算，不负责打印；draw_big_ball只负责在指定位置画一个球，不关心为何要画。这样的函数更容易被复用。

通过函数参数或配置文件来控制函数的行为，而不是硬编码。例如，draw_frame和draw_fake_GUI等函数都接受GameState对象作为参数，根据其中的rows和cols动态调整绘制大小，使其能够适应5x5、10x10等不同规模的棋盘，而不是为每种尺寸都写一个函数。

6. 附件：源程序

注意排版格式，对齐方式，可以采用两列排版，正文采用宋体，小五，行距为单倍行距

不需要全部源程序，main函数、菜单选择、输入输出错误等常规内容建议舍弃，挑部分核心代码贴上来即可。

(笔者注：调整了制表符占位，以及行距，应该能令观感更佳)

```
void draw_big(const GameState& game, int part, int mode)
{
    if (part == 0) {
        cout << " ";
        return;
    }
    if (part == 1)
        cout << " ";
    if (part == 2 && mode == 1)
        cout << " | O | ";
    else if (part == 2 && mode == -1)
        cout << " | X | ";
    if (part == 3)
        cout << " ";
}

void draw_big_ball(GameState& game, int x, int y, int i, int j)
{
    cct_gotoxy(x + i * 8, y + j * 4 - 1);
    draw_big(game, 1, 1);
    cct_gotoxy(x + i * 8, y + j * 4);
    draw_big(game, 2, 1);
    cct_gotoxy(x + i * 8, y + j * 4 + 1);
    draw_big(game, 3, 1);
    cct_setcolor();
}
```

```

void draw_big_cross(GameState& game, int x, int y, int i, int j)
{
    cct_gotoxy(x + i * 8, y + j * 4 - 1);
    draw_big(game, 1, -1);
    cct_gotoxy(x + i * 8, y + j * 4);
    draw_big(game, 2, -1);
    cct_gotoxy(x + i * 8, y + j * 4 + 1);
    draw_big(game, 3, -1);
    cct_setcolor();
}

void loop_BIG(GameState& game) // 有分隔线
{
    cct_enable_mouse();
    cct_setcursor(CURSOR_INVISIBLE);

    int hint_cols = 0;
    for (int j = 0; j < game.rows; j++) {
        if (game.row_hint_count[j] > hint_cols) {
            hint_cols = game.row_hint_count[j];
        }
    } // 预留列数

    int hint_rows = 0;
    for (int j = 0; j < game.cols; j++) {
        if (game.col_hint_count[j] > hint_rows) {
            hint_rows = game.col_hint_count[j];
        }
    } // 预留行数

    int mx, my, maction, keycode1, keycode2;
    const int x = hint_cols * 2 + 8;
    const int y = hint_rows + 5;
    while (true) {
        int ret = cct_read_keyboard_and_mouse(mx, my, maction, keycode1, keycode2);
        int grid_oldx = -1024;
        int grid_oldy = -1024;

        int grid_x = -1024; // 单独处理 0 附近的截断向 0 取整问题
        if (mx - hint_cols * 2 - 8 >= 0)
            grid_x = (mx - hint_cols * 2 - 8) / 8;

        int grid_y = -1024; // 还是要截断处理
        if (my - hint_rows - 4 >= 0)
            grid_y = (my - hint_rows - 4) / 4;

        // 计算在格子内的相对位置
        int cell_x = (mx - hint_cols * 2 - 8) % 8;
        int cell_y = (my - hint_rows - 4) % 4;

        if (ret == CCT_MOUSE_EVENT) {
            cct_showstr(9, hint_rows + game.rows * 4 + 5, "[当前光标] ");
            if (grid_x >= 0 && grid_x < game.cols && grid_y >= 0 && grid_y < game.rows &&
                cell_x >= 0 && cell_x <= 5 && cell_y >= 0 && cell_y <= 2) {
                if (maction == MOUSE_ONLY_MOVED && (grid_x != grid_oldx || grid_y != grid_oldy)) {

                    char r[2];
                    r[0] = 'A' + grid_y;
                    r[1] = '\0';
                    char l[2];
                    l[0] = 'a' + grid_x;
                    l[1] = '\0';
                }
            }
        }
    }
}

```

```

cct_showstr(20, hint_rows + game.rows * 4 + 5, r);
cct_showstr(21, hint_rows + game.rows * 4 + 5, "行");
cct_showstr(23, hint_rows + game.rows * 4 + 5, 1);
cct_showstr(24, hint_rows + game.rows * 4 + 5, "列");
grid_oldx = grid_x;
grid_oldy = grid_y;
}
else if (maction == MOUSE_LEFT_BUTTON_CLICK) {
    cct_showstr(9, hint_rows + game.rows * 4 + 5, "[读到左键]");
    char r[2];
    r[0] = 'A' + grid_y;
    r[1] = '\0';
    char l[2];
    l[0] = 'a' + grid_x;
    l[1] = '\0';
    cct_showstr(20, hint_rows + game.rows * 4 + 5, r);
    cct_showstr(21, hint_rows + game.rows * 4 + 5, "行");
    cct_showstr(23, hint_rows + game.rows * 4 + 5, 1);
    cct_showstr(24, hint_rows + game.rows * 4 + 5, "列");
    if (game.player[grid_y][grid_x] != 1) {
        cct_gotoxy(mx - mx % 2, my);

        if (game.cheat_mode == false || (game.cheat_mode == true && game.grid[grid_y][grid_x]
== 1)) {
            cct_setcolor(COLOR_HBLUE, 0);
        }
        else if (game.cheat_mode == true && game.grid[grid_y][grid_x] == 0) {
            cct_setcolor(COLOR_HRED, 0);
        }

        draw_big_ball(game, x, y, grid_x, grid_y);
        cct_setcolor();
        game.player[grid_y][grid_x] = 1; // 选中
    }
    else {
        cct_gotoxy(mx - mx % 2, my);

        if (game.cheat_mode == false || (game.cheat_mode == true && game.grid[grid_y][grid_x]
== 0)) {
            cct_setcolor(COLOR_HWHITE, COLOR_BLACK);
            draw_big_space(game, x, y, grid_x, grid_y);
        }
        else if (game.cheat_mode == true && game.grid[grid_y][grid_x] == 1) {
            cct_setcolor(COLOR_WHITE, COLOR_BLACK);
            draw_big_ball(game, x, y, grid_x, grid_y);
        }

        cct_setcolor();
        game.player[grid_y][grid_x] = 0; // 恢复
    }
}
else if (maction == MOUSE_RIGHT_BUTTON_CLICK) {
    cct_showstr(9, hint_rows + game.rows * 4 + 5, "[读到右键]");
    if (game.player[grid_y][grid_x] != -1) {
        cct_gotoxy(mx - mx % 2, my);

        if (game.cheat_mode == false) {
            cct_setcolor(COLOR_HRED, COLOR_BLACK);
        }
        else if ((game.cheat_mode == true && game.grid[grid_y][grid_x] == 1)) {

```

```

    cct_setcolor(COLOR_HBLUE, COLOR_BLACK);
}
else if (game.cheat_mode == true && game.grid[grid_y][grid_x] == 0) {
    cct_setcolor(COLOR_HRED, COLOR_BLACK);
}

draw_big_cross(game, x, y, grid_x, grid_y);
cct_setcolor();
game.player[grid_y][grid_x] = -1;
}
else {
    cct_gotoxy(mx - mx % 2, my);

    if (game.cheat_mode == true && game.grid[grid_y][grid_x] == 1) {
        cct_setcolor(COLOR_WHITE, COLOR_BLACK);
        draw_big_ball(game, x, y, grid_x, grid_y);
    }
    else {
        cct_setcolor(COLOR_HWHITE, 0);
        draw_big_space(game, x, y, grid_x, grid_y);
    }

    cct_setcolor();
    game.player[grid_y][grid_x] = 0;
}
}
}
else {
    cct_showstr(20, hint_rows + game.rows * 4 + 5, "位置非法");
}
}
else if (ret == CCT_KEYBOARD_EVENT) {
    if (keycode1 == 'y' || keycode1 == 'Y') {
        cct_showstr(9, hint_rows + game.rows * 4 + 5, "[读到提交键]");
        if (check(game)) {
            Sleep(500);
            cct_showstr(9, hint_rows + game.rows * 4 + 5, "[提交成功]");
        }
        break;
    }
    else {
        Sleep(500);
        cct_showstr(9, hint_rows + game.rows * 4 + 5, "提交错误, 可用作弊模式查看");
    }
}
else if (keycode1 == 'z' || keycode1 == 'Z') {
    if (game.cheat_mode == false) {
        cct_showstr(9, hint_rows + game.rows * 4 + 5, "[作弊模式开]");
        game.cheat_mode = true;
        Sleep(500);
        for (int i = 0; i < game.rows; i++) {
            for (int j = 0; j < game.cols; j++) {
                if (game.grid[i][j] == 1 && game.player[i][j] == 0) {
                    cct_setcolor(COLOR_WHITE, COLOR_BLACK); // 1. 有球没选圈
                    draw_big_ball(game, x, y, j, i);
                }
                else if (game.grid[i][j] == 1 && game.player[i][j] == 1) {
                    cct_setcolor(COLOR_HBLUE, COLOR_BLACK); // 2. 有球已选圈
                    draw_big_ball(game, x, y, j, i);
                }
                else if (game.grid[i][j] == 0 && game.player[i][j] == 1) {

```

```

        cct_setcolor(COLOR_HRED, COLOR_BLACK); // 3. 无球已选圈
        draw_big_ball(game, x, y, j, i);
    }
    else if (game.grid[i][j] == 0 && game.player[i][j] == -1) {
        cct_setcolor(COLOR_HRED, COLOR_BLACK); // 4. 无球已选叉
        draw_big_cross(game, x, y, j, i);
    }
    else if (game.grid[i][j] == 1 && game.player[i][j] == -1) {
        cct_setcolor(COLOR_HBLUE, COLOR_BLACK); // 5. 有球已选叉
        draw_big_cross(game, x, y, j, i);
    }
    else {
        cct_setcolor(COLOR_HWHITE, COLOR_BLACK); // 6. 啥也没有
        draw_big_space(game, x, y, j, i);
    }
}
}
cct_setcolor();
}
else {
    cct_showstr(9, hint_rows + game.rows * 4 + 5, "[作弊模式关]");
    game.cheat_mode = false;
    Sleep(500);
    for (int i = 0; i < game.rows; i++) {
        for (int j = 0; j < game.cols; j++) {
            if (game.player[i][j] == 1) {
                cct_setcolor(COLOR_HBLUE, COLOR_BLACK); // 已选圈
                draw_big_ball(game, x, y, j, i);
            }
            else if (game.player[i][j] == -1) {
                cct_setcolor(COLOR_HRED, COLOR_BLACK); // 已选叉
                draw_big_cross(game, x, y, j, i);
            }
            else {
                cct_setcolor(COLOR_HWHITE, COLOR_BLACK); // 啥也没有
                draw_big_space(game, x, y, j, i);
            }
        }
    }
    cct_setcolor();
}
}
else if (keycode1 == 'q' || keycode1 == 'Q') {
    cct_showstr(9, hint_rows + game.rows * 4 + 5, "[读到 Q/q] 游戏结束");
    break;
}
else {
    cct_showstr(9, hint_rows + game.rows * 4 + 5, "[读到键码] ");
    cout << keycode1 << "/" << keycode2 << " ";
}
}
}

cct_disable_mouse();
cct_setcursor(CURSOR_SHOWING);
}

```